

Segurança da Informação

Prof. Claudinei Dias (Ney)

N1 - Vulnerabilidades em Aplicações Web (SaaS)

Pentesting (parte 1)

Desenvolver uma versão 1.0 (simples e sem recursos de segurança) de uma API RESTful usando Node.js que permita a autenticação e autorização de usuários. A API será responsável por gerenciar operações básicas de CRUD (Create, Read, Update, Delete) para usuários.

Especificações para API RESTful (CRUD para Usuários)

GET: Listar todos os usuários registrados.

POST: Criar novos usuários no sistema.

PUT: Atualizar os dados de um usuário existente.

DELETE: Excluir um usuário do sistema.

A API deve seguir o padrão REST, retornando os status adequados para cada operação (por exemplo, 200 OK, 201 Created, 404 Not Found, etc.).

Utilizar um banco de dados MySQL ou MongoDB Atlas para armazenar as informações dos usuários.

Pentesting (parte 2)

SQL/NoSQL Injection

Testar este Ataque:

- No campo de nome de usuário, insira um comando SQL/NoSQL malicioso para obter todas as informações da tabela de usuários.
- Envie o formulário e observe o comportamento.

Cross-Site Scripting (XSS)

Executar este Ataque na Versão 1 desenvolvida na parte 1:

- Injete um código JavaScript malicioso.
- Envie o formulário e observe o comportamento no navegador.

Cross-Site Request Forgery (CSRF)

Testar este Ataque:

- Crie uma página HTML maliciosa em um servidor controlado pelo atacante.
- Nesta página, inclua um formulário com uma solicitação POST para realizar uma ação maliciosa em um aplicativo da web alvo. Por exemplo, enviar uma solicitação para alterar a senha do usuário.
- Engane um usuário autenticado para visitar essa página maliciosa.

Pentesting (parte 3)

Desenvolver uma versão 2.0 (com recursos de prevenção para obter a segurança da informação) com proteção de rotas específicas por meio de autenticação via JWT (JSON Web Token) e ações para correção sugeridas nos pentesting executados acima.

As rotas devem ser implementadas usando o framework Express.

Especificação para Autenticação e Autorização (sistema de login e registro de usuários)

Utilizar JWT (JSON Web Token) para autenticação de usuários. O token JWT deve ser gerado durante o login e enviado no cabeçalho de todas as requisições protegidas.

Proteger as rotas de PUT e DELETE para que apenas usuários autenticados possam acessá-las.

Rotas protegidas devem retornar 401 Unauthorized se o token não for válido ou estiver ausente.

Utilizar a biblioteca bcrypt para criptografar senhas durante o registro de novos usuários.

1) Rota de Registro:

POST /register. Rota para registrar novos usuários. Os campos obrigatórios são username, email e password. A senha deve ser criptografada antes de ser armazenada no banco de dados.

2) Rota de Login:

POST /login. Rota para login de usuários. O sistema deve validar as credenciais e, em caso de sucesso, retornar um token JWT que será usado para autenticação nas próximas requisições.

3) Rotas CRUD para Usuários:

GET /users: Retorna uma lista de todos os usuários (disponível para qualquer usuário autenticado).

PUT /users: Atualiza os dados de um usuário com base em seu id (apenas usuários autenticados).

DELETE /users: Exclui um usuário com base em seu id (apenas usuários autenticados).

4) Proteção de Rotas:

As rotas PUT e DELETE devem ser protegidas, ou seja, somente usuários com um token JWT válido devem ter acesso.

5) SQL/NoSQL Injection

Ações para Correção e Prevenção:

- Use consultas parametrizadas ou *prepared statements* para evitar a inserção direta de dados do usuário em comandos SQL/NoSQL.
- Realize a validação de entrada para garantir que apenas dados válidos sejam inseridos nos comandos SQL/NoSQL.
- Limite as permissões de banco de dados para que o aplicativo da web tenha acesso apenas aos recursos necessários.

6) Cross-Site Scripting (XSS)

Ações para Correção e Prevenção:

- Valide e sanitize todos os dados de entrada do usuário.
- Use cabeçalhos de segurança, como Content Security Policy (CSP), para mitigar ataques XSS.
- Configure cookies de sessão com o atributo HTTPOnly para proteger contra roubo de cookies.

7) Cross-Site Request Forgery (CSRF)

Ações para Correção e Prevenção:

- Use tokens CSRF (também conhecidos como tokens anti-CSRF) para proteger solicitações sensíveis contra ataques CSRF.
- Certifique-se de que todas as operações sensíveis, como alterar a senha ou realizar transações financeiras, exijam um token CSRF válido.

8) Implementar o versionamento de código com Git e incluir um arquivo README explicando como rodar o projeto localmente. O código deve seguir boas práticas de organização e legibilidade.

ENTREGA:

OBS.: Deve ser realizado em times.

Código-fonte completo da API hospedado em um repositório no GitHub.

Arquivo README.md contendo:

Instruções para instalação das dependências (via npm ou yarn).

Explicação de como rodar a aplicação localmente.

Exemplos de requisições usando ferramenta Postman.

Relatórios (Versão em Sumário Executivo e Versão Técnica).